

```
In [1]: # funcname(input1, input2)

# concatenate / vector
x = c(1,3,2,5)
y = c(1,3,2,5)
x

?funcname # opens documentation on function

# add 2 sets of same length numbers (paired by index)
length(x)
length(y)
x+y

# same as terminal
ls()
rm(x,y)
ls()

rm(list=ls()) # rfemove all at once
```

1 · 3 · 2 · 5

4

4

2 · 6 · 4 · 10

'x' · 'y'

No documentation for 'funcname' in specified packages and libraries: you could try
'??funcname'

In []:

```
In [2]: ?matrix
x=matrix(data=c(1,2,3,4),nrow=2,ncol=2)
x
```

A

matrix:

2 × 2

of

type

dbl

1 3

2 4

Matrices

Description

`matrix` creates a matrix from the given set of values.

`as.matrix` attempts to turn its argument into a matrix.

`is.matrix` tests if its argument is a (strict) matrix.

Usage

```
matrix(data = NA, nrow = 1, ncol = 1, byrow = FALSE,  
       dimnames = NULL)
```

```
as.matrix(x, ...)  
## S3 method for class 'data.frame'  
as.matrix(x, rownames.force = NA, ...)
```

```
is.matrix(x)
```

Arguments

data an optional data vector (including a list or `expression` vector). Non-atomic classed **R** objects are coerced by `as.vector` and all attributes discarded.

nrow the desired number of rows.

ncol the desired number of columns.

byrow logical. If `FALSE` (the default) the matrix is filled by columns, otherwise the matrix is filled by rows.

dimnames a `dimnames` attribute for the matrix: `NULL` or a `list` of length 2 giving the row and column names respectively. An empty list is treated as `NULL`, and a list of length one as row names. The list can be named, and the list names will be used as names for the dimensions.

x an **R** object.

... additional arguments to be passed to or from methods.

`rownames.force`

logical indicating if the resulting matrix should have character (rather than `NULL`) rownames. The default, `NA`, uses `NULL` rownames if the data frame has 'automatic' row.names or for a zero-row data frame.

Details

If one of `nrow` or `ncol` is not given, an attempt is made to infer it from the length of `data` and the other parameter. If neither is given, a one-column matrix is returned.

If there are too few elements in `data` to fill the matrix, then the elements in `data` are recycled. If `data` has length zero, `NA` of an appropriate type is used for atomic vectors (`0` for raw vectors) and `NULL` for lists.

`is.matrix` returns `TRUE` if `x` is a vector and has a `"dim"` attribute of length 2 and `FALSE` otherwise. Note that a `data.frame` is **not** a matrix by this test. The function is generic: you can write methods to handle specific classes of objects, see `InternalMethods`.

`as.matrix` is a generic function. The method for data frames will return a character matrix if there is only atomic columns and any non- (numeric/logical/complex) column, applying `as.vector` to factors and `format` to other non-character columns. Otherwise, the usual coercion hierarchy (logical < integer < double < complex) will be used, e.g., all-logical data frames will be coerced to a logical matrix, mixed logical-integer will give a integer matrix, etc.

The default method for `as.matrix` calls `as.vector(x)`, and hence e.g. coerces factors to character vectors.

When coercing a vector, it produces a one-column matrix, and promotes the names (if any) of the vector to the rownames of the matrix.

`is.matrix` is a primitive function.

The `print` method for a matrix gives a rectangular layout with dimnames or indices. For a list matrix, the entries of length not one are printed in the form 'integer,7' indicating the type and length.

Note

If you just want to convert a vector to a matrix, something like

```
dim(x) <- c(nx, ny)
dimnames(x) <- list(row_names, col_names)
```

will avoid duplicating `x` and preserve `class(x)` which may be useful, e.g., for `Date` objects.

References

Becker RA, Chambers JM, Wilks AR (1988). *The New S Language*. Chapman and Hall/CRC, London. ISBN 053409192X.

See Also

`data.matrix`, which attempts to convert to a numeric matrix.

A matrix is the special case of a two-dimensional `array`. `inherits(m, "array")` is true for a `matrix` `m`.

Examples

```
is.matrix(as.matrix(1:10))
!is.matrix(warpbreaks) # data.frame, NOT matrix!
warpbreaks[1:10,]
as.matrix(warpbreaks[1:10,]) # using as.matrix.data.frame(.) method
```

```
## Example of setting row and column names
mdat <- matrix(c(1,2,3, 11,12,13), nrow = 2, ncol = 3, byrow = TRUE,
               dimnames = list(c("row1", "row2"),
                               c("C.1", "C.2", "C.3")))
mdat
```

[Package *base* version 4.6.1]

```
In [3]: x=matrix(c(1,2,3,4),2,2)
x
```

```
A
matrix:
2 x 2
of
type
dbl
1 3
2 4
```

for data processed horizontally/row by row

not assigned to a variable: prints but not saved

```
In [4]: matrix(c(1,2,3,4),2,2,byrow=TRUE)
```

```
A
matrix:
2 x 2
of
type
dbl
1 2
3 4
```

```
In [5]: sqrt(x)
x^2
```

```
A matrix: 2 x 2 of type
dbl
```

```
1.000000 1.732051
1.414214 2.000000
```

```
A
matrix:
2 x 2 of
type
dbl
1 9
4 16
```

```
In [6]: x=rnorm(50) # default mean = 0, sd = 1
y=x+rnorm(50,mean=50,sd=.1)
cor(x,y)
```

```
0.996061433104265
```

seed & rnorm: sets the starting point in the infinite sequence of numbers, pulls the next n numbers from that portion of the sequence, reshaping them into normal distribution

calling rnorm again moves through the sequence

```
In [7]: # reproducibility
set.seed(1303) # same set of random numbers
rnorm(50)
```

-1.14397631447974 · 1.34212936561501 · 2.18539047574276 ·
 0.536392517923731 · 0.0631929664685468 · 0.502234482468979 ·
 -0.000416724686432643 · 0.565819840539162 · -0.572522688962623 ·
 -1.11022500727697 · -0.0486871233624514 · -0.695656217619366 ·
 0.828917480303335 · 0.206652855081802 · -0.235674509102427 ·
 -0.556310491381104 · -0.364754357080585 · 0.862355034263622 ·
 -0.630771535367711 · 0.313602125215739 · -0.931495317661392 ·
 0.823867618473952 · 0.523370702077482 · 0.706921411979056 ·
 0.420204325601679 · -0.269052154682033 · -1.51031729990999 ·
 -0.69021247657504 · -0.143471952443572 · -1.0135274099044 ·
 1.57327373614751 · 0.0127465054882014 · 0.872647049887217 ·
 0.422066190530336 · -0.0188157916578866 · 2.61574896890584 ·
 -0.693140174826871 · -0.266321780991085 · -0.720636441231524 ·
 1.36773420645149 · 0.264007332160512 · 0.632186807367191 ·
 -1.33065098578719 · 0.0268888182209596 · 1.0406363207788 ·
 1.31202379854711 · -0.0300020766733214 · -0.250025712488174 ·
 0.0234144856913592 · 1.65987065574227

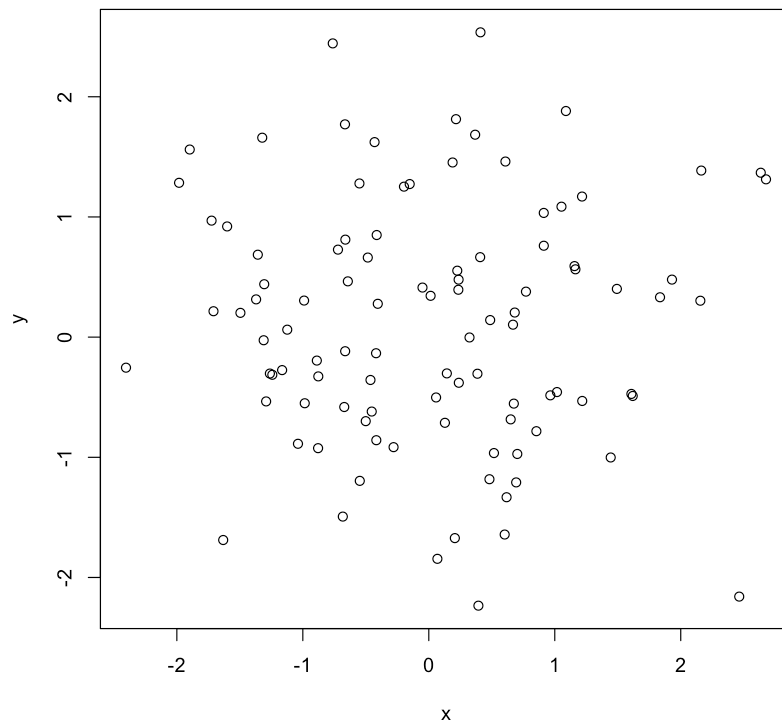
```

In [8]: set.seed(3)
        y=rnorm(100)
        mean(y)
        var(y)
        sqrt(var(y)) # variance squared = standard dev
        sd(y)
  
```

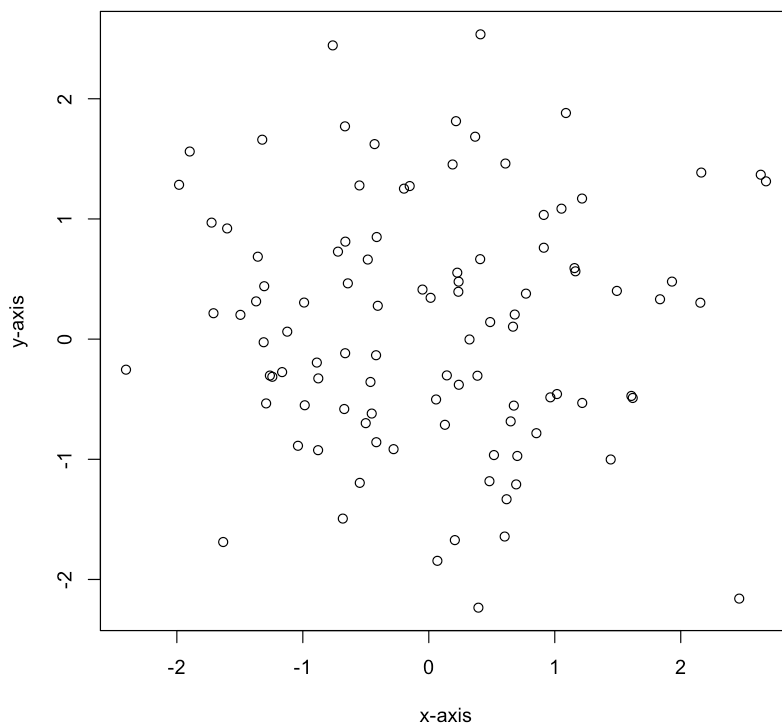
0.0110355710943716
 0.732867501277449
 0.856076808047881
 0.856076808047881

```

In [9]: x=rnorm(100)
        y=rnorm(100)
        plot(x,y)
        plot(x,y,xlab="x-axis",ylab="y-axis",main="plot of x vs y")
  
```



plot of x vs y



```
In [10]: pdf("Figure.pdf")  
plot(x,y,col="green")  
dev.off() # done creating plot
```

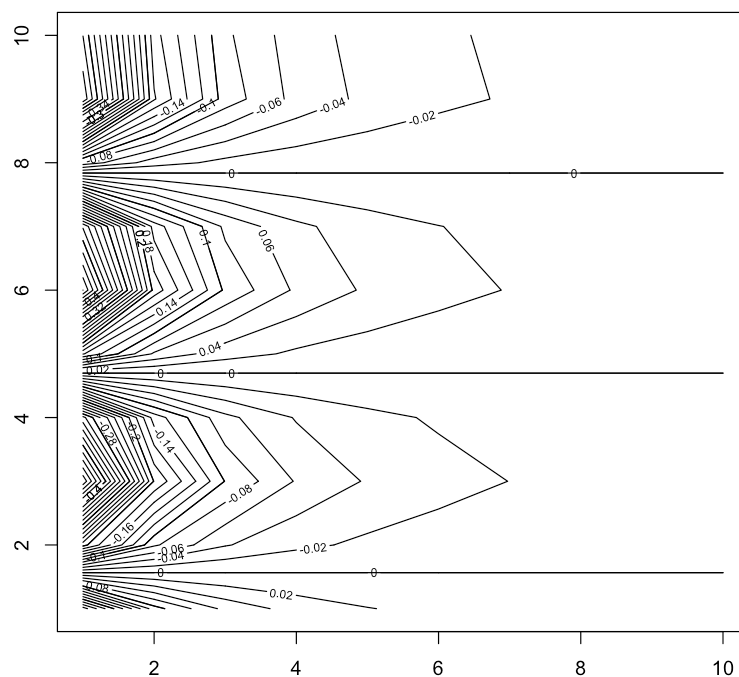
agg_record_10ef2ff07dc4: 2

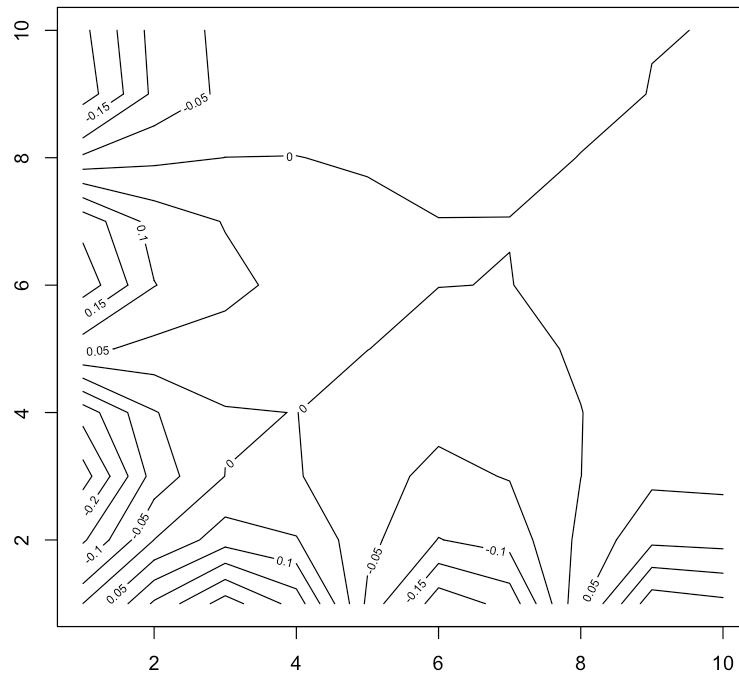
```
In [11]: # seq(a,b) vector of integers between a & b
# seq(0,1,length=10) 10 equal spaced #s between

x=seq(1,10)
x
x=1:10 # splicing
xx=seq(-pi, pi, length=50)
```

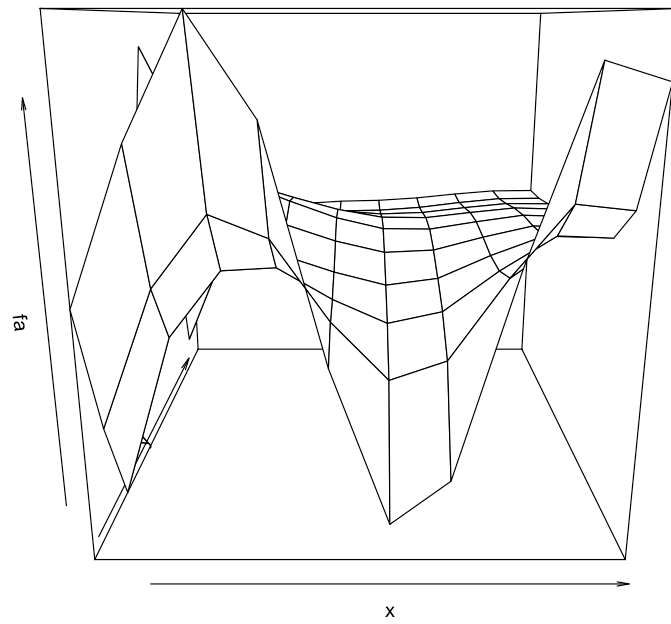
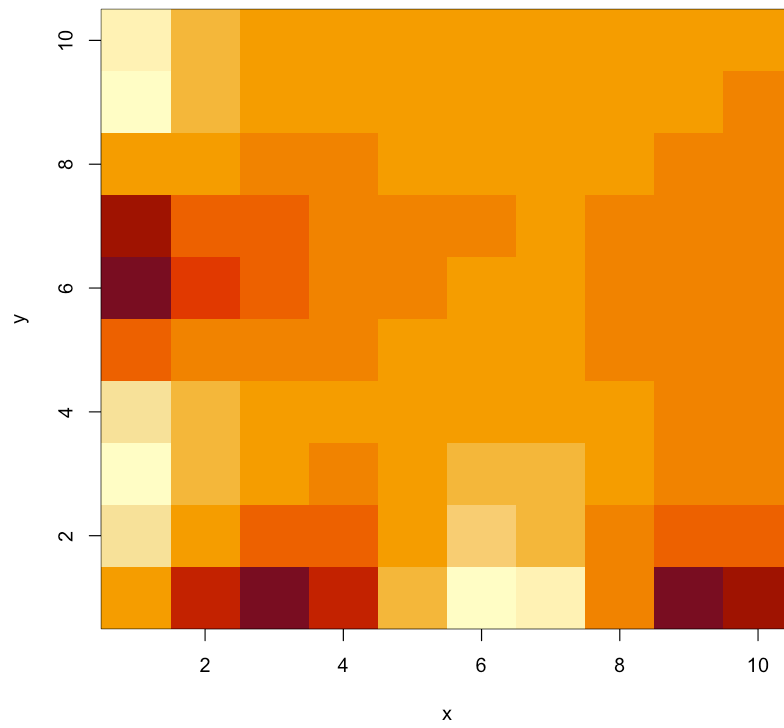
1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9 · 10

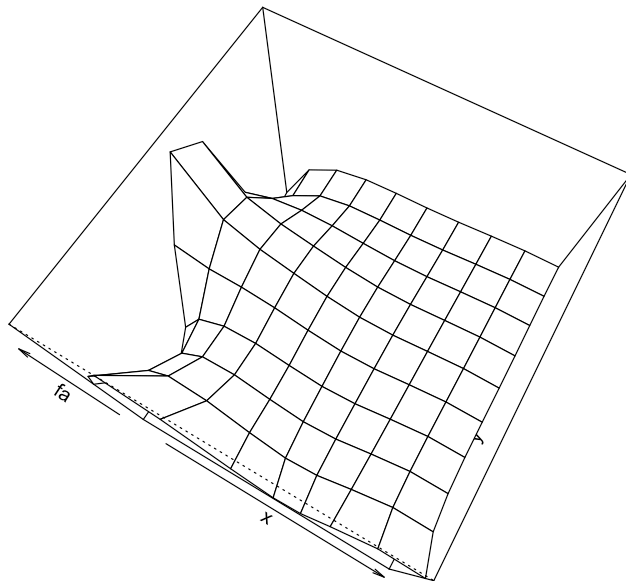
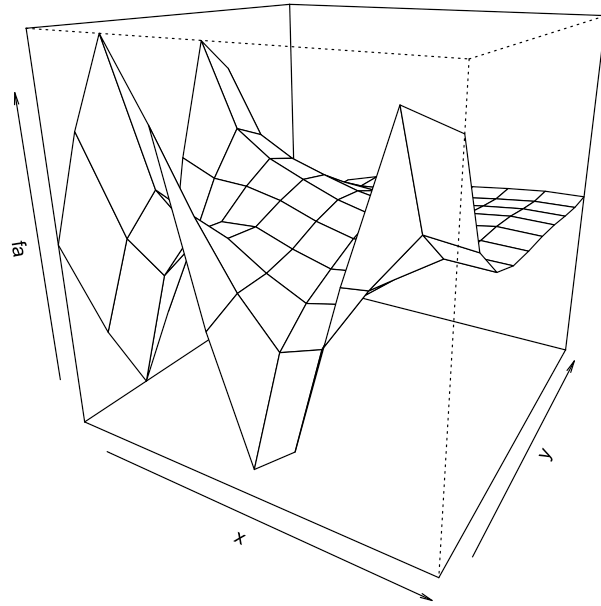
```
In [12]: # 3d data, contour map (topographical)
# (vectorX, vY, matrix Z for each pair of (x,y) coords)
y=x
f=outer(x,y,function(x,y)cos(y)/(1+x^2))
contour(x,y,f)
contour(x,y,f,nlevels=45,add=T)
fa=(f-t(f))/2
contour(x,y,fa,nlevels=15)
```

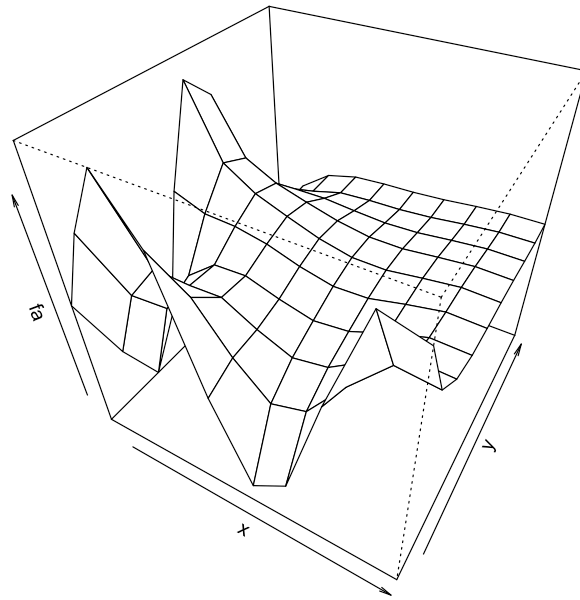




```
In [13]: # image() heatmap - same as contour, color-coded plot based on z value
# persp() 3d plot, theta & phi for viewing angles
image(x,y,fa)
persp(x,y,fa)
persp(x,y,fa,theta=30,phi=20)
persp(x,y,fa,theta=30,phi=70)
persp(x,y,fa,theta=30,phi=40)
```







```
In [14]: A=matrix(1:16,4,4)
A
A[2,3] # [row, col]

# splice by passing in vectors
A[c(1,3), c(2,4)]
A[1:3,2:4] # elements at rows 1-3 & cols 2-4

A[1,]# select all

A[-c(1,3),] # select all except

dim(A) # num rows, num cols
```

A matrix: 4 × 4
of type int

1 5 9 13

2 6 10 14

3 7 11 15

4 8 12 16

10

A
matrix:
2 × 2 of
type int
5 13
7 15

A matrix: 3
× 3 of type
int
5 9 13
6 10 14
7 11 15

1 · 5 · 9 · 13

A matrix: 2 × 4
of type int
2 6 10 14
4 8 12 16

4 · 4

```
In [15]: # write.table() # export

Auto=read.table(file="/Users/emilyahmad/DataScience/OM065-Data-Science

# fix(Auto) # doesn't work well outside Rstudio, opens spreadsheet in
# edit(Auto) # also didn't work
install.packages("DT")
library(DT)
datatable(Auto)
```

The downloaded binary packages are in
/var/folders/rk/hm976cm116b00fkw44hrkkgc0000gn/T//RtmpWf9hF4/do
wnloaded_packages

V1	V2	V3	V4	V5	V6	V7	V8	V9
----	----	----	----	----	----	----	----	----

```
In [16]: # alternative to fix(Auto)
library(IRdisplay)
library(htmlwidgets)

my_widget <- datatable(head(Auto), filter = "top")

temp_file <- tempfile(fileext = ".html")
saveWidget(my_widget, temp_file, selfcontained = TRUE)

display_html(file = temp_file)
```

V1	V2	V3	V4	V5	V6	V7	V8	V9
All	All	All	All	All	All	All	All	All

```
In [30]: ## assigns first line as variable names
## characters treated as missing elements
Auto=read.table("Auto.data", header=T, na.strings="?")
# fix(Auto)
```

```
In [18]: Auto=read.csv("Auto.csv",header=T,na.strings="?")
dim(Auto) # dimensions 397 observations (rows), 9 variables (cols)
Auto[1:4,]
```

397 · 9

A data.frame: 4 × 9

	mpg	cylinders	displacement	horsepower	weight	acceleration	year	origin
	<dbl>	<int>	<dbl>	<int>	<int>	<dbl>	<int>	<int>
1	18	8	307	130	3504	12.0	70	
2	15	8	350	165	3693	11.5	70	
3	18	8	318	150	3436	11.0	70	
4	16	8	304	150	3433	12.0	70	

```
In [19]: Auto=na.omit(Auto) # removes rows containing missing observations
dim(Auto)
names(Auto) # variable names
```

392 · 9

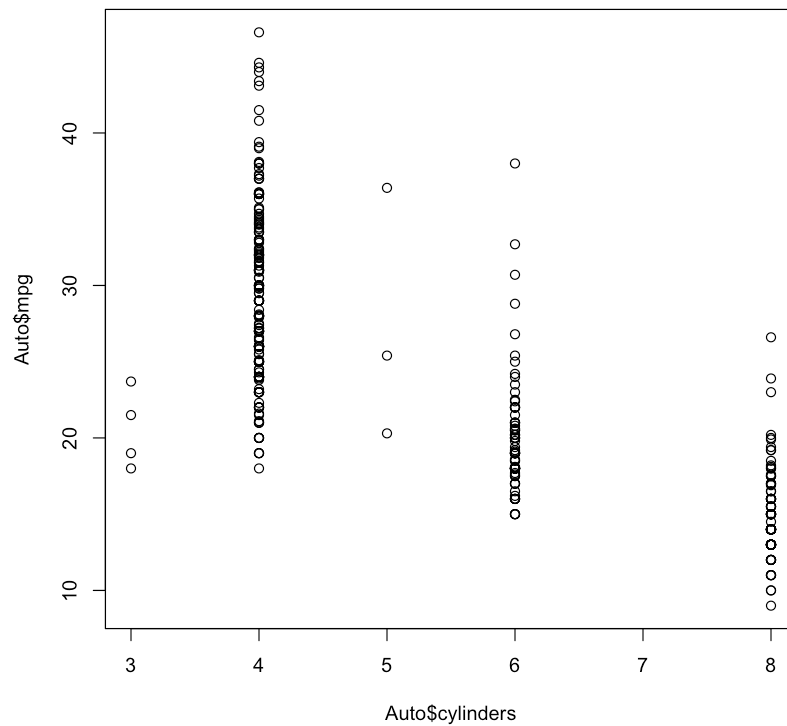
```
'mpg' · 'cylinders' · 'displacement' · 'horsepower' · 'weight' · 'acceleration' · 'year' ·
'origin' · 'name'
```

```
In [20]: # cannot simply type var name, R doesn't search in Auto dataset

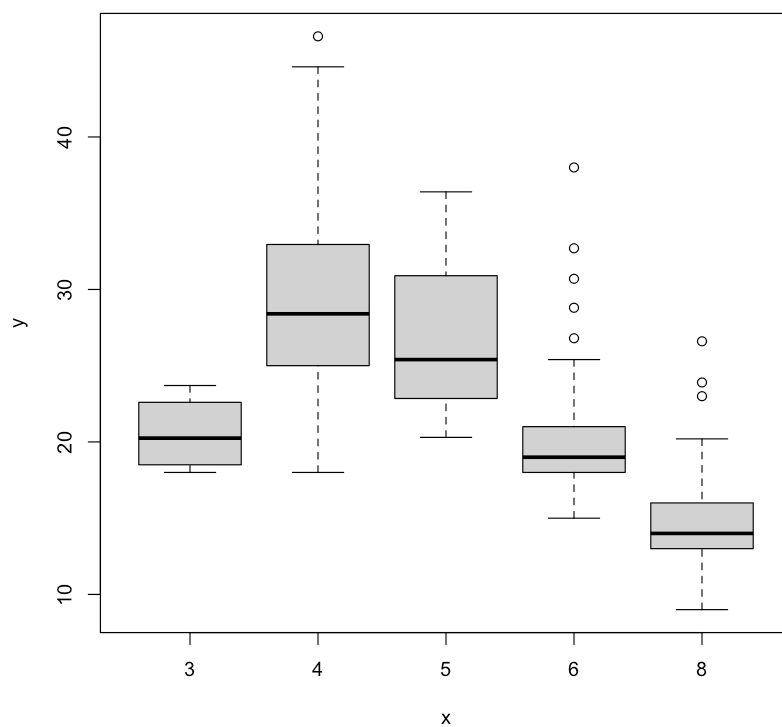
# cylinders var stored in numeric vector, R treats as quantitative
plot(Auto$cylinders, Auto$mpg)

attach(Auto)
display_html(file = temp_file)
```

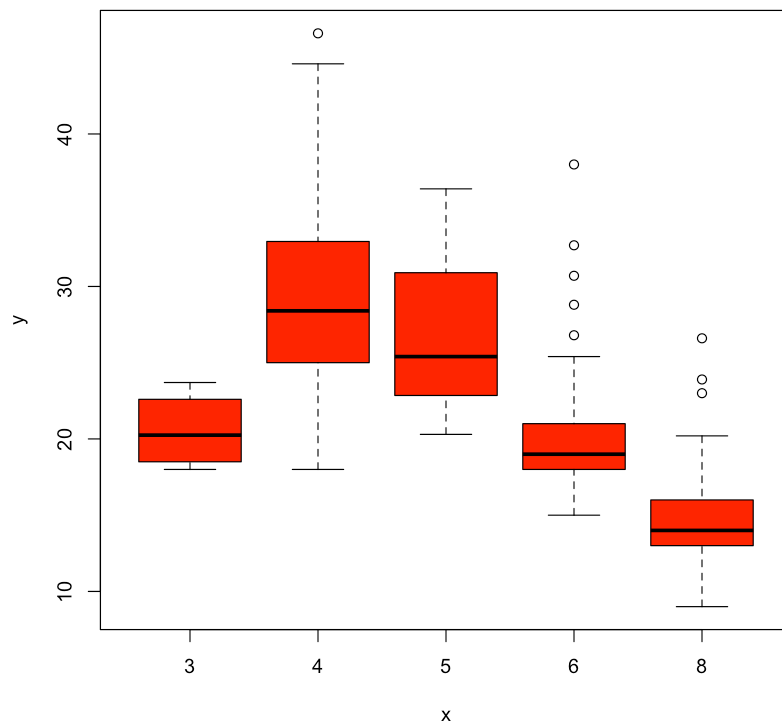
V1	V2	V3	V4	V5	V6	V7	V8	V9
All	All	All	All	All	All	All	All	All



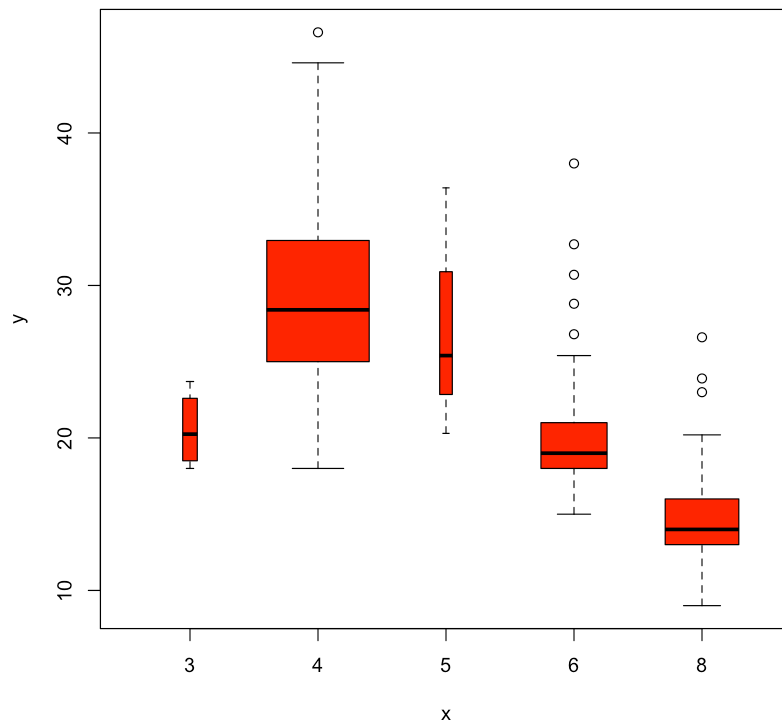
```
In [21]: # converts quantitative to qualitative var
cylinders=as.factor(cylinders)
# categorical x-axis: boxplots (rather than scatterplot for quantitati
plot(cylinders, mpg)
```



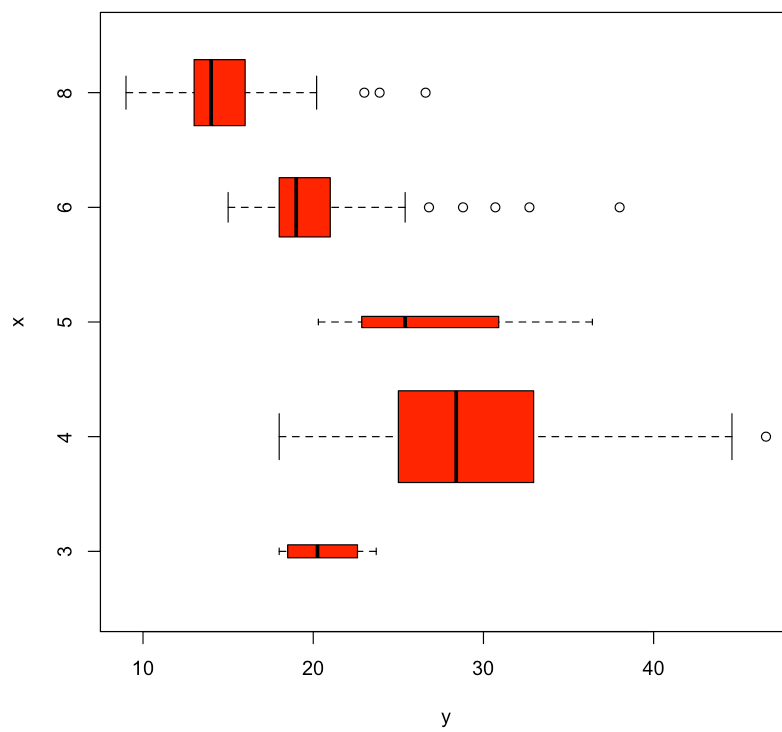
```
In [22]: # customization  
plot(cylinders, mpg, col="red")
```



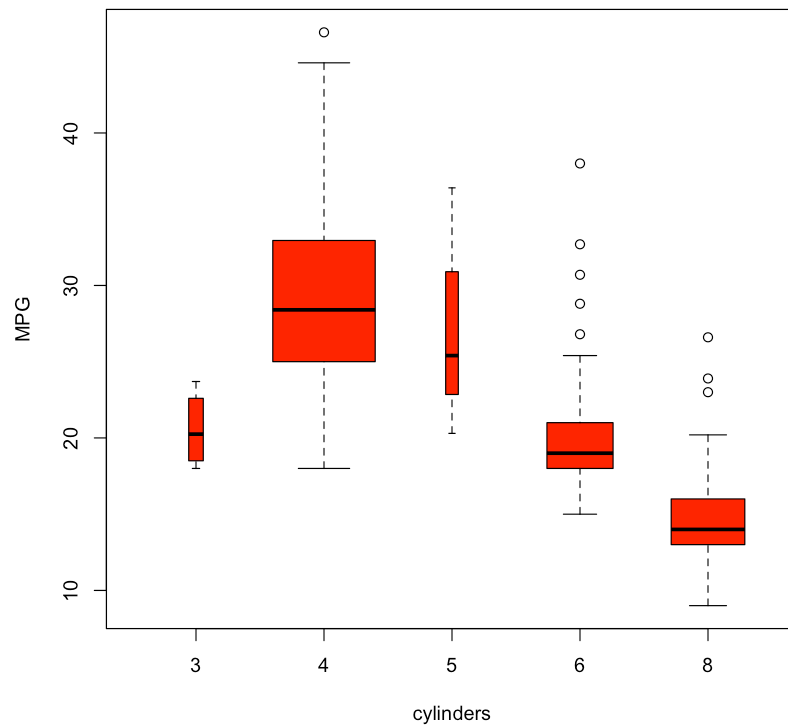
```
In [23]: plot(cylinders, mpg, col="red", varwidth=T)
```



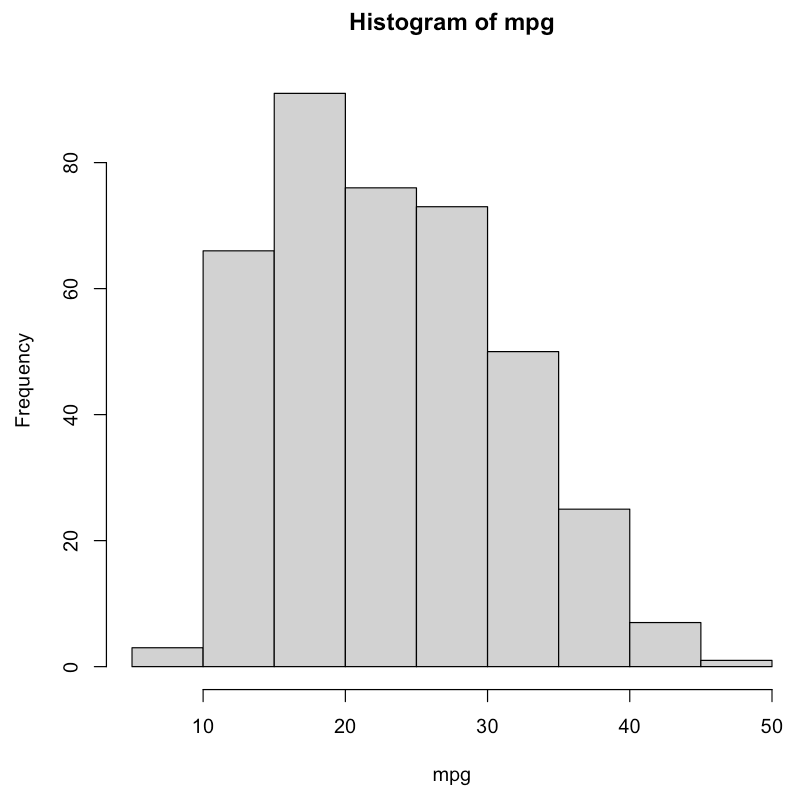
```
In [24]: plot(cylinders, mpg, col="red", varwidth=T, horizontal=T)
```



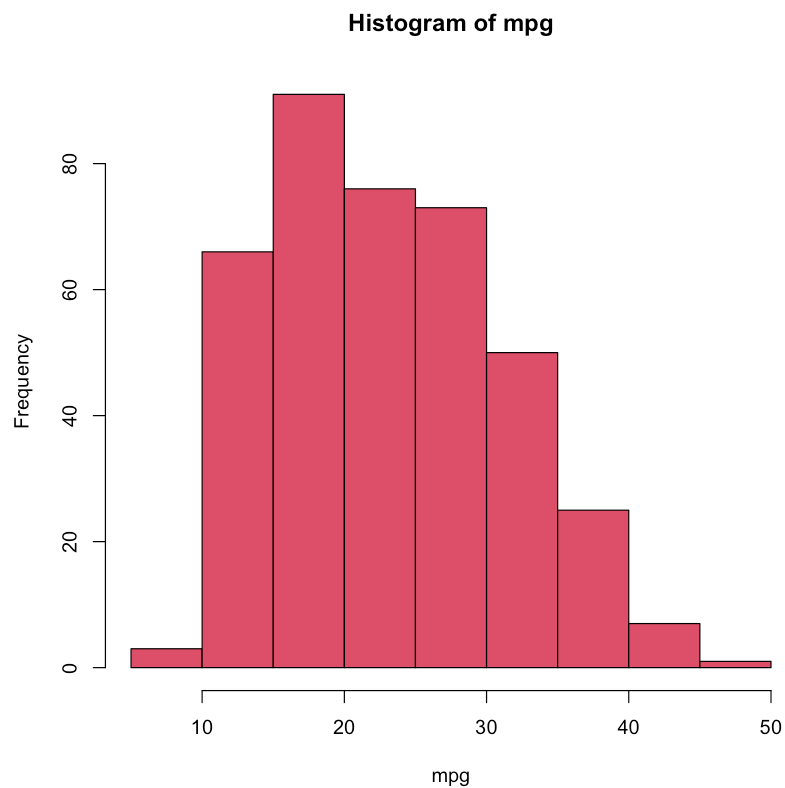
```
In [25]: plot(cylinders, mpg, col="red", varwidth=T, xlab="cylinders", ylab="MP
```



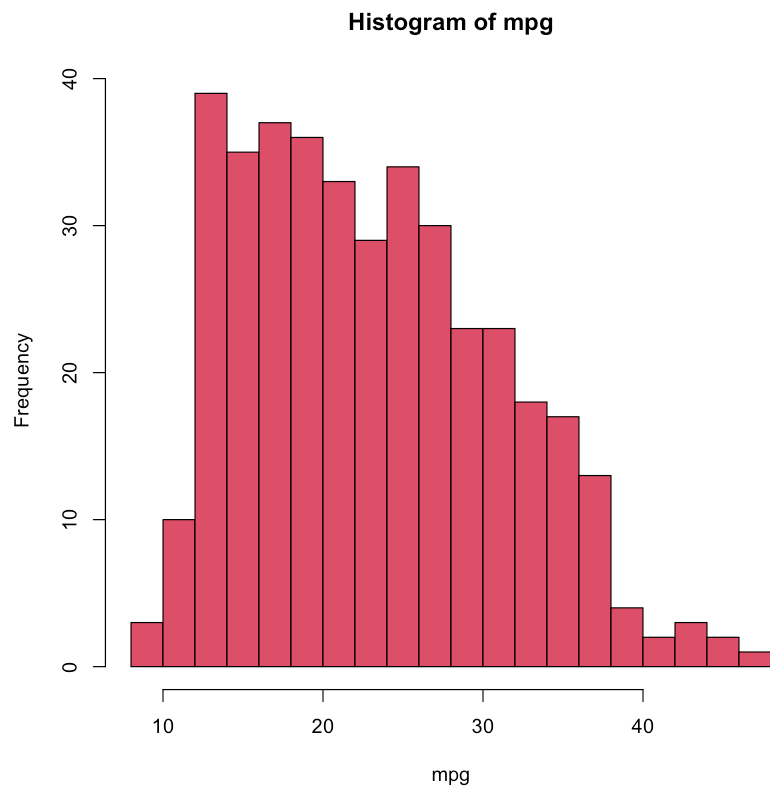
```
In [26]: hist(mpg)
```



```
In [27]: hist(mpg,col=2)
```

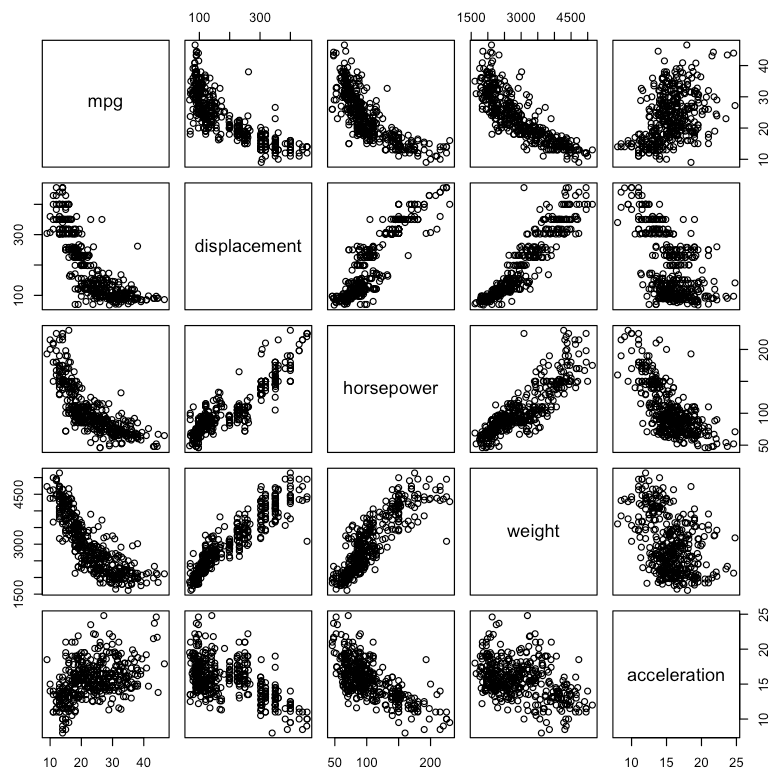


```
In [28]: hist(mpg,col=2,breaks = 15)
```

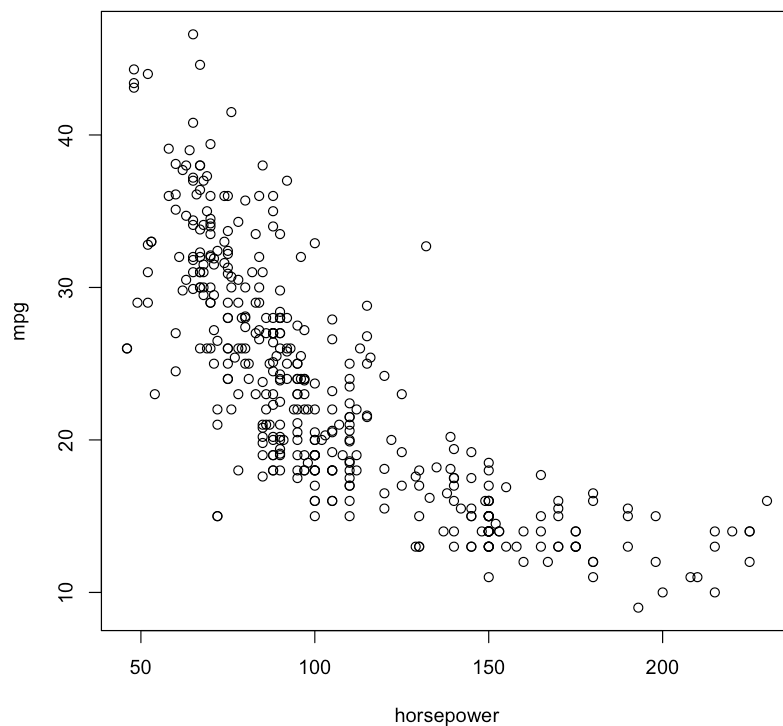


```
In [31]: # didn't work first time, Auto was non-numeric (from as.factor() comma  
# convert back to qualitative - still didn't work, entire dataset is c  
cylinders=as.character(cylinders)
```

```
# pairs(Auto) # scatterplot matrix for every pair of vars in a dataset
pairs(~ mpg + displacement + horsepower + weight + acceleration, Auto)
```



```
In [32]: plot(horsepower, mpg)
identify(horsepower, mpg, name) # works best in rstudio
```



In [33]: `summary(Auto)`

```

      mpg      cylinders      displacement      horsepower      weight
Min.   : 9.00   Min.   :3.000   Min.   : 68.0   Min.   : 46.0   Min.
1st Qu.:17.50   1st Qu.:4.000   1st Qu.:104.0   1st Qu.: 75.0   1st Q
Median :23.00   Median :4.000   Median :146.0   Median : 93.5   Median
Mean   :23.52   Mean   :5.458   Mean   :193.5   Mean   :104.5   Mean
3rd Qu.:29.00   3rd Qu.:8.000   3rd Qu.:262.0   3rd Qu.:126.0   3rd Q
Max.   :46.60   Max.   :8.000   Max.   :455.0   Max.   :230.0   Max.

      acceleration      year      origin      NAs      name
Min.   : 8.00   Min.   :70.00   Min.   :1.000   Length :397
1st Qu.:13.80   1st Qu.:73.00   1st Qu.:1.000   N.unique:304
Median :15.50   Median :76.00   Median :1.000   N.blank : 0
Mean   :15.56   Mean   :75.99   Mean   :1.574   Min.nchar: 6
3rd Qu.:17.10   3rd Qu.:79.00   3rd Qu.:2.000   Max.nchar: 36
Max.   :24.80   Max.   :82.00   Max.   :3.000

```

In [34]: `summary(mpg)`

```

      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
      9.00  17.00   22.75   23.45  29.00   46.60

```

```

In [35]: # q() to shut down
         # savehistory() records commands

```